

Diseño y análisis de algoritmos

Ricardo Wehbe

UADE

9 de agosto de 2026

Programa

- 1 Introducción. Paisaje general
- 2 Algoritmos
- 3 Complejidad temporal
 - Recurrencia
 - Notaciones asintóticas
 - P & NP
- 4 Ejercicios propuestos

¿Dónde estamos?

- 1 Introducción. Paisaje general
- 2 Algoritmos
- 3 Complejidad temporal
 - Recurrencia
 - Notaciones asintóticas
 - P & NP
- 4 Ejercicios propuestos

Welcome back my friends to the show that never ends

Emerson, Lake & Palmer, *Karn Evil 9*

Las reglas del juego

- Hay dos exámenes parciales. El primero de ellos abarcará complejidad, *divide-and-conquer* y programación dinámica. El segundo abarcará complejidad (sí, de nuevo), y algoritmos *greedy* y *backtracking*. Se puede recuperar uno y sólo uno de ellos.
- La cursada se aprueba habiendo aprobado los dos parciales o uno de ellos y el recuperatorio correspondiente.
- Hay ejercicios sugeridos. Hay una explicación en la siguiente diapositiva de cómo funciona esto.
- Para quienes hubieren aprobado ambos parciales, existe la posibilidad de rendir el examen final anticipadamente el día del recuperatorio.
- Hay un examen final regular.

Algunas malas noticias

- Ésta es una materia difícil.
- ¿Se puede aprender a nadar en un aula oyendo hablar a un instructor?
- Tampoco a programar.
- Algunos problemas observados:
 - Lógica. Hay una cierta deficiencia de pensamiento lógico.
 - Análisis. Se tiende a resolver problemas mecánicamente sin buscar llegar primero a una comprensión del problema.
 - Síntesis. No hay un buen relacionamiento entre los temas.
- ¿Y por qué me cuentan todo esto?
- Porque justamente esto es lo que se evalúa.

¿De qué trata esta materia?

- Estudio de diferentes técnicas de construcción de algoritmos para encontrar la más eficiente desde el punto de vista de la complejidad temporal para un problema dado.
- Análisis de eficiencia temporal.

Nuestro objeto de estudio son entonces los *algoritmos*.

Las técnicas que estudiaremos son (por orden de aparición):

- *Divide and conquer*
- Algoritmos “voraces” (*Greedy algorithms*)
- Programación dinámica
- *Backtracking*

¿Dónde estamos?

- 1 Introducción. Paisaje general
- 2 Algoritmos
- 3 Complejidad temporal
 - Recurrencia
 - Notaciones asintóticas
 - P & NP
- 4 Ejercicios propuestos

¿Qué es un algoritmo?

- Primer intento de formalización: David Hilbert, a comienzos del siglo XX (“ein Verfahren”, un procedimiento.)
- Un algoritmo es una secuencia ordenada y finita de acciones que transforman un conjunto de datos de entrada en datos de salida y que resuelven un problema específico.
- Un algoritmo tiene cero o más datos de entrada, puede o no tener una salida, cada paso está unívocamente determinado y debe terminar en un número finito de pasos.
- Estas definiciones son para esta materia; en general, algunos algoritmos pueden no terminar y en algunos casos cada paso puede *no* estar unívocamente determinado.

Corrección y eficiencia de algoritmos. Informalmente

- Corrección: ¿resuelve el algoritmo nuestro problema?
- Eficiencia: ¿cuán rápidamente lo hace?
- Si para contar vacas les contamos las patas y dividimos el resultado por cuatro, estamos resolviendo el problema. Pero no de la manera más eficiente.
- El algoritmo de ordenamiento por selección es 3 veces más lento que el MergeSort si tomamos 10 elementos. Es 750 veces más lento si queremos ordenar 1000 elementos.

Corrección de algoritmos. Formalmente

- Programa: escritura de un algoritmo en un lenguaje que pueda ser ejecutado por una computadora (Java, Python, &c.)
- Instancia: una instancia de un programa es un valor de entrada válido. Un programa puede tener numerosas instancias (en general, infinitas)
- Corrección: un programa es correcto (o eficaz) si funciona de la manera esperada para *todas* las instancias de un problema.

¿“Todas” las instancias?

- ¿Produce el siguiente programa siempre un número primo para $n \geq 0$?

```
1.  algoritmo primo ( $n : \text{int}$ ) {  
2.      return  $n^2 + n + 41$ ;  
3.  }
```

- Respuesta: averigüenlo para la próxima clase.

¿Dónde estamos?

- 1 Introducción. Paisaje general
- 2 Algoritmos
- 3 Complejidad temporal
 - Recurrencia
 - Notaciones asintóticas
 - P & NP
- 4 Ejercicios propuestos

No hay en la tierra una sola página, una sola palabra que [sea simple], ya que todas postulan el universo, cuyo más notorio atributo es la complejidad

J. L. Borges, *El informe de Brodie*

Eficiencia temporal. Principio de invariancia

Conjetura (Principio de invariancia)

Dado un algoritmo A y dos implementaciones de éste, I_1 e I_2 , donde para una entrada de longitud n la primera tarda $T_1(n)$ unidades y la segunda $T_2(n)$ unidades, entonces existe un número real c y un entero n_0 tales que

$$T_1(n) \leq cT_2(n) \quad \text{para todo } n > n_0$$

En otras palabras, dos implementaciones distintas del *mismo* algoritmo no difieren sino en una constante multiplicativa.

¿Qué nos dice el principio de invariancia?

- “Dos implementaciones” pueden ser dos implementaciones en diferentes lenguajes o la misma implementación en dos máquinas diferentes.
- Lo que implica este principio (que, recordémoslo, es una conjetura) es que la relación entre los tiempos de ejecución entre las dos implementaciones para diferentes tamaños de la entrada se mantendrá aproximadamente constante.
- Esto pone, en definitiva, un límite al aumento de eficiencia que se puede conseguir utilizando solamente tecnología.
- En otras palabras, la tecnología no puede compensar completamente las debilidades de un algoritmo ineficiente.

Mejor y peor caso

- Mejor caso: el menor de los costos de todas las instancias de un problema.
- Caso promedio: el costo promedio de todas las instancias de un problema.
- Peor caso: el mayor de los costos de todas las instancias de un problema.
- Trabajaremos siempre, salvo que se diga explícitamente lo contrario, con el peor caso.

Mejor y peor caso: un ejemplo

Considere el siguiente algoritmo, que busca un número x en un arreglo no ordenado T .

```
1.  algoritmo Buscar ( $T$  : array[0.. $n-1$ ] of int,  $x$  : int)
2.  int  $i = 0$ ;
3.  while ( $i < n$ ) &&  $T[i] \neq x$ )
4.       $i++$ ;
5.  if ( $i < n$ )
6.      return  $i$ 
5.  else
6.      return  $-1$ 
```

- ¿Cuál es el mejor caso? $x = T[0]$; sólo se entra una vez al ciclo
- ¿Cuál es el peor caso? El número no está o $x = T[n-1]$; se entra n veces al ciclo
- ¿Cuál es el caso promedio? Se ingresa al ciclo $n/2$ veces

Operaciones elementales

Definición (Operaciones elementales)

Una operación elemental es una operación cuyo tiempo de ejecución máximo puede ser limitado por una constante que depende sólo de la implementación utilizada.

En otras palabras, una operación elemental tiene una duración constante. Esta constante varía según la implementación del lenguaje utilizado.

Cálculo del número de operaciones elementales

- Para una secuencia de operaciones, la cantidad total de operaciones es la suma de operaciones de la secuencia.
- Para un condicional, la cantidad de operaciones elementales es **1** más el máximo entre la cantidad de operaciones de ambas ramas de la condición.
- Para un ciclo, la cantidad de operaciones elementales es **N** veces la cantidad de operaciones elementales del bloque de repetición, donde **N** es la cantidad de repeticiones del ciclo.
- Para una llamada a un método, el número de operaciones elementales es **1** más el número de operaciones elementales de la ejecución del método.
- ¿Qué pasa en el caso de recurrencia?

¿Dónde estamos?

- 1 Introducción. Paisaje general
- 2 Algoritmos
- 3 Complejidad temporal
 - Recurrencia
 - Notaciones asintóticas
 - P & NP
- 4 Ejercicios propuestos

La mer, la mer toujours recommencée

(El mar, el mar siempre comenzado.)

P. Valéry, *Le Cimetière marin*

Algoritmos recursivos

- Son algoritmos que se llaman a sí mismos.
- Debe haber al menos un *caso base*, que se resuelve sin llamada recursiva.
- Para cualquier valor válido de entrada, en cada nuevo llamado la distancia al caso base debe disminuir.

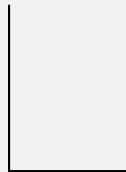
Algoritmos recursivos: un ejemplo (factorial)

```
1.  algoritmo factorial( $n$  : int)
2.    if ( $n == 0$ )
3.      return 1;
4.    else
5.      return  $n * \text{factorial}(n - 1)$ ;
```

- ¿Hay caso base?
- ¿Disminuye la distancia al caso base para todos los valores válidos de n ?

¿Cómo funciona un algoritmo recursivo?

`factorial(5) = 120`



Pila del sistema

¿Cómo se construye un algoritmo recursivo?

- ¿Cuál es la estructura del problema? Tenemos

$$\text{factorial}(n) = n * (n - 1) * (n - 2) * \dots * 3 * 2 * 1$$

- ¿Cuáles son los casos base? Claramente, $n = 0$ y, opcionalmente, $n = 1$.
- ¿Es una solución para $\text{factorial}(n - 1)$ útil para obtener una solución de $\text{factorial}(n)$? Por supuesto; basta mirar la definición:

$$\begin{aligned}\text{factorial}(n) &= n * (n - 1) * (n - 2) * \dots * 3 * 2 * 1 \\ &= n * \underbrace{\text{factorial}(n - 1)}_{\text{parte recursiva}}\end{aligned}$$

- Ahora juntemos todo:

```
1.  algoritmo factorial (int n)
2.      if (n == 0 || n == 1)
3.          return 1
4.      else
5.          return n * factorial(n - 1)
```

Algoritmos recursivos: un ejemplo (búsqueda binaria)

```
1.  algorithm binrec (T : array[0..n - 1] of int, ini, fin, x : int)
2.  if (ini > fin) {
3.      return -1;
4.  } else {
5.      mid = (ini + fin)/2
6.      if (x == T[mid])
7.          return mid;
8.      else if (x < T[mid])
9.          return binrec(T, ini, mid - 1, x);
10.     else
11.         return binrec(T, mid + 1, fin, x);
12. }
```

- ¿Hay caso base?
- ¿Disminuye la distancia al caso base para todos los valores válidos de *n*?

¿Dónde estamos?

- 1 Introducción. Paisaje general
- 2 Algoritmos
- 3 Complejidad temporal
 - Recurrencia
 - Notaciones asintóticas
 - P & NP
- 4 Ejercicios propuestos

¿Para qué sirven las notaciones asintóticas

- Vimos anteriormente cómo calcular una cota superior a la cantidad de operaciones elementales que se realizan durante la ejecución de un algoritmo.
- Pero no nos interesa entrar en tanto detalle. Supongamos que tenemos un algoritmo A_1 que ejecuta $3x^3 + 7x^2 + 18$ operaciones elementales y un algoritmo A_2 que efectúa un máximo de $0,5x^3$ operaciones elementales.
- Claramente, A_2 es más eficiente que A_1 . Pero, para la notación asintótica, los dos están en la misma categoría: ambos juegan en la liga de los algoritmos de costo cúbico.
- Para esto, ignoraremos los términos de menor crecimiento (en el caso de A_1 , los términos $7x^2$ y 18) y las constantes multiplicativas (3 para A_1 y $0,5$ para A_2 .)
- Por supuesto, esto es sólo una aproximación. En los cálculos de complejidad, nos conformamos con esto.

Las notaciones $\mathcal{O}$ y Θ

- Notación: $\mathbb{R}^+$ es el conjunto de los reales *no negativos*.
- Si $f, g : \mathbb{N} \rightarrow \mathbb{R}^+$, entonces $f(n) \in \mathcal{O}(g(n))$ (el “orden” de f es g) si existen constantes c_1 y n_0 tales que

$$0 \leq f(n) \leq c_1 g(n) \text{ para todo } n \geq n_0$$

- Si $f, g : \mathbb{N} \rightarrow \mathbb{R}^+$, entonces $f(n) \in \Theta(g(n))$ (el “orden exacto” de f es g) si existen constantes c_1 , c_2 y n_0 tales que

$$0 \leq c_2 g(n) \leq f(n) \leq c_1 g(n) \text{ para todo } n \geq n_0$$

¿Y qué significa todo esto?

- Se dice que una función $f(n)$ está (por ejemplo) en $\mathcal{O}(n)$, si se puede encontrar alguna constante c_2 tal que a partir de cierto valor $c_2 n$ queda siempre por encima de $f(n)$.
- Por ejemplo, $7n + 18 \in \mathcal{O}(n)$, ya que si elegimos $c_2 = 50$, para cualquier valor mayor que cero $50n > 7n + 18$.
- Observe que $n^2 \notin \mathcal{O}(n)$; por grande que sea c_2 , siempre hay un valor a partir del cual $n^2 > c_2 n$. Por ejemplo, si $c_2 = 1\,000\,000$, a partir de $n = 1\,000\,001$ el cuadrado supera a la función lineal.
- En la notación $\mathcal{O}$ ignoramos las constantes ($f_1(n) = 3n^2$ y $f_2(n) = 9n^2$ están ambas en $\mathcal{O}(n^2)$) y las funciones de crecimiento menor ($f_3(n) = 5n^2 + 17n + 4$ y $f_4(n) = 9n^2 + 122n - 1$ están ambas en $\mathcal{O}(n^2)$)
- Lo mismo vale para la notación Θ .
- Observe que $\mathcal{O}(g(n))$, lo mismo que $\Theta(g(n))$ representan un *conjunto* de funciones.

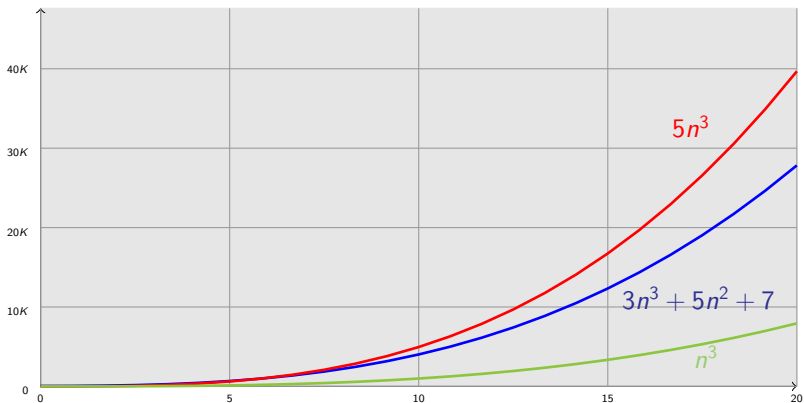
Un ejemplo. Primera parte

- El objetivo es mostrar que $f(n) = 3n^3 + 5n^2 + 7 \in \Theta(n^3)$.
- Para ello, debemos encontrar las constantes c_1 , c_2 y n_0 tales que

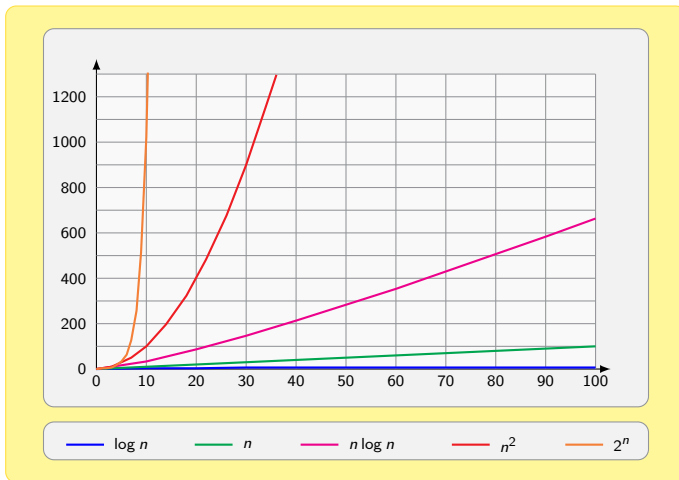
$$c_2 n^3 \leq 3n^3 + 5n^2 + 7 \leq c_1 n^3 \text{ para todo } n \geq n_0$$

- Por ejemplo, $c_2 = 1$, $c_1 = 5$ y $n_0 = 3$.

Un ejemplo. Segunda parte



Algunas funciones de crecimiento



Resolución de recurrencias. Caso de substracción

- Es una función del tipo

$$T(n) = \begin{cases} c & \text{si } 0 \leq n < b \\ aT(n-b) + p(n) & \text{si } n \geq b \end{cases}$$

- Donde a es la cantidad de llamadas recursivas (peor caso), b es la cantidad de unidades en que disminuye la entrada en cada paso recursivo (peor caso) y k es el grado del polinomio $p(n)$, que son las sentencias que se ejecutan fuera del llamado recursivo.
- Entonces tenemos:

$$T(n) \in \begin{cases} \Theta(n^k) & \text{si } a < 1 \\ \Theta(n^{k+1}) & \text{si } a = 1 \\ \Theta(n^k a^{n/b}) & \text{si } a > 1 \end{cases}$$

Resolución de recurrencias. Caso de división

- Es una función del tipo

$$T(n) = \begin{cases} c & \text{si } 0 \leq n < b \\ aT(n/b) + p(n) & \text{si } n \geq b \end{cases}$$

- Donde a es la cantidad de llamadas recursivas (peor caso), b es la cantidad de unidades en que se divide la entrada en cada paso recursivo (peor caso) y k es el grado del polinomio $p(n)$, que son las sentencias que se ejecutan fuera del llamado recursivo.
- Entonces tenemos:

$$T(n) \in \begin{cases} \Theta(n^k) & \text{si } a < b^k \\ \Theta(n^k \log n) & \text{si } a = b^k \\ \Theta(n^{\log_b a}) & \text{si } a > b^k \end{cases}$$

Ejemplo de resolución (caso de substracción)

```
1.  algoritmo factorial(n : int)
2.      if n = 0
3.          return 1;
4.      else
5.          return n * factorial(n - 1);
```

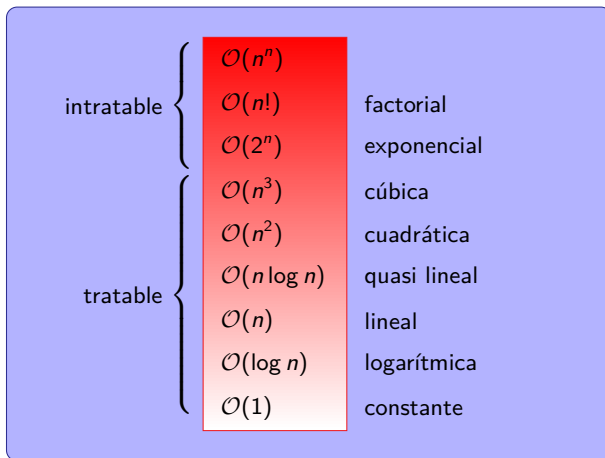
- Aquí tenemos: $a = 1$ $b = 1$ $k = 0$
- Por lo tanto: $T(n) \in \Theta(n^{k+1}) = \Theta(n)$

Ejemplo de resolución (caso de división)

```
1.  algorithm binrec (T : array[0..n - 1] of int, ini, fin, x : int)
2.  if (ini > fin) {
3.      return -1;
4.  } else {
5.      mid = (ini + fin)/2
6.      if (x == T[mid])
7.          return mid;
8.      else if (x < T[mid])
9.          return binrec(T, ini, mid - 1, x);
10.     else
11.         return binrec(T, mid + 1, fin, x);
12. }
```

- Aquí tenemos: $a = 1$ $b = 2$ $k = 0$
- Por lo tanto $T(n) \in \Theta(n^k \log n) = \Theta(\log n)$

Jerarquía (parcial) de casos de complejidad



Complejidad. Una síntesis

- Un algoritmo en general tarda más si el tamaño de la entrada n crece.
- Queremos determinar cómo influye el tamaño de la entrada en el tiempo de ejecución.
- Las notaciones $\mathcal{O}$ y Θ nos dan una medida aproximada de esta relación.
- Ambas notaciones ignoran las constantes: si un algoritmo tarda $4n$ y el otro $7n$, ambos están en el mismo orden de complejidad, $\mathcal{O}(n)$ (o $\Theta(n)$).
- Ambas notaciones ignoran los términos menos significativos: si un algoritmo tarda $2n^2 + 3n - 5$ y el otro $7n^2 + 18n + 144$, nos quedamos con los términos más importantes, o sea con $2n^2$ y $7n^2$ y, por lo que se dijo arriba, ambos están en el mismo orden de complejidad, $\mathcal{O}(n^2)$ (o $\Theta(n^2)$).

¿Dónde estamos?

- 1 Introducción. Paisaje general
- 2 Algoritmos
- 3 Complejidad temporal
 - Recurrencia
 - Notaciones asintóticas
 - P & NP
- 4 Ejercicios propuestos

Problemas polinomiales y exponenciales

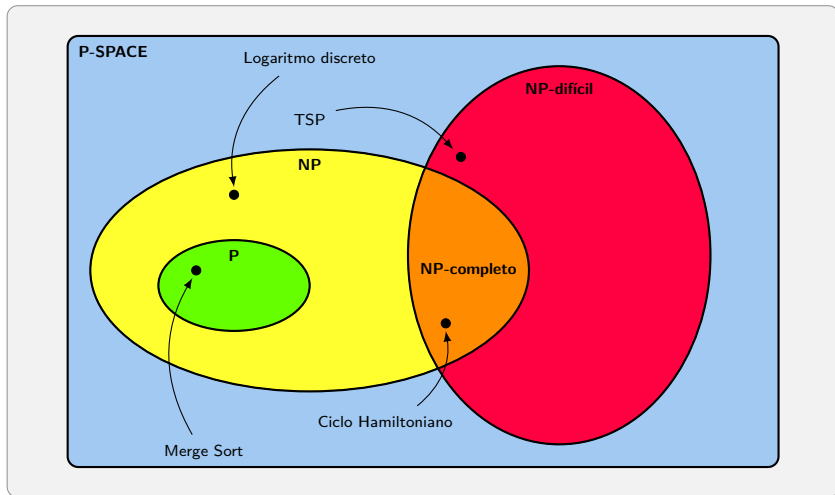
Complejidad polinomial

Heap Sort
Merge Sort
Bubble Sort
Búsqueda binaria
Producto de matrices
Factorial
Agregar a un Heap
Comparación de conjuntos
Producto escalar

Complejidad exponencial

Ciclo Hamiltoniano
TSP
Mochila 0-1
SAT
Suma de subconjuntos
Logaritmo Discreto
Camarillas
Cobertura de vértices
Conjunto independiente

Algunas clases de complejidad



¿Dónde estamos?

- 1 Introducción. Paisaje general
- 2 Algoritmos
- 3 Complejidad temporal
 - Recurrencia
 - Notaciones asintóticas
 - P & NP
- 4 Ejercicios propuestos

Ejercicios propuestos 1

- 1 “Este algoritmo se ejecuta por lo menos en $\mathcal{O}(n^2)$ ”. ¿Qué opinión le merece esta frase?
- 2 El algoritmo A_1 resuelve un problema en n^2 días y el algoritmo A_2 lo resuelve en n^3 segundos. ¿A partir de qué tamaño de la entrada el algoritmo A_1 supera al algoritmo A_2 ? ¿Cuánto demora la solución en este caso?
- 3 ¿Es cierto que $n^2 \in \mathcal{O}(n^3)$? ¿Y que $n^2 \in \Theta(n^3)$?
- 4 Calcule la complejidad de los algoritmos A , B , C y D , que se dan en las siguientes diapositivas.

Problemas propuestos: Algoritmo A

```
1.  Algoritmo AlgoA(int n)
2.      if  $n \geq 2$ 
3.          bool a = getValue(n); //  $getValue \in \Theta(n^2)$ 
4.          if a {
5.              print (n);
6.              AlgoA(n/2);
7.          } else {
8.              AlgoA(n/2);
9.          }
```

Problemas propuestos: Algoritmo *B*

```
1.  Algoritmo AlgoB(int n)
2.      if  $n \geq 2$  {
3.          bool a = getValue(n)    // getValue  $\in \Theta(1)$ 
4.          if a {
5.              CalculateCost(n);    // CalculateCost  $\in \Theta(n^2)$ 
6.              AlgoB(n/2);
7.          } else {
8.              AlgoB(n/2);
9.          }
10.     } else {
11.         AlgoB(n/2);
12.     }
```

Problemas propuestos: Algoritmo C

```
1.  Algoritmo AlgoC(int n)
2.      if  $n \geq 2$  {
3.          bool a = getValue(n);    //  $getValue \in \Theta(1)$ 
4.          if a {
5.              CalculateCost(n);    //  $CalculateCost \in \Theta(n^2)$ 
6.              AlgoC(n/2);
7.          } else {
8.              AlgoC(n/2);
9.          }
10.     } else {
11.         AssessRisk(n);            //  $AssessRisk \in \Theta(n)$ 
12.         AlgoC(n/2);
13.         CalculateCost(n);        //  $CalculateCost \in \Theta(n^2)$ 
14.         AlgoC(n/2);
15.     }
```

Problemas propuestos: Algoritmo *D*

```
1.  Algoritmo AlgoD(int n,int u)
2.      if  $n \leq 0$  {
3.          return u
4.      endif
5.      for ( $i = 0; i \leq n; i++$ )
6.          int v = Calcular(n)      // Calcular  $\in \Theta(??)$ 
7.          AlgoD(n - 1, v)
8.      endfor
```

Ejercicios propuestos 2

- 5 Suponga que tiene que elegir entre los siguientes algoritmos para resolver un problema:
- El algoritmo 1 resuelve el problema dividiéndolo en cinco subproblemas de la mitad de tamaño, resolviendo recursivamente cada subproblema y combinando las soluciones en tiempo lineal.
 - El algoritmo 2 resuelve un problema de tamaño n resolviendo recursivamente dos problemas de tamaño $n - 1$ y combinándolos en tiempo constante.
 - El algoritmo 3 resuelve un problema de tamaño n dividiéndolo en nueve subproblemas de tamaño $n/3$, resolviendo recursivamente cada subproblema y combinando las soluciones en tiempo cuadrático.

¿Cuál es la complejidad de cada uno de estos algoritmos y cuál elegiría? Recuerde que, en los casos en que no es evidente, puede tomarse límite para $n \rightarrow \infty$ para desempatar entre dos algoritmos.

Ejercicios propuestos 3

- 6 Considere el algoritmo *Mystery* de la siguiente diapositiva.
 - ¿Cuál es la complejidad temporal de este algoritmo?
 - Proponga un algoritmo alternativo que haga lo mismo con una complejidad temporal estrictamente menor. No es necesario conservar el orden de entrada del arreglo *A*.
- 7 Calcule la complejidad temporal de los algoritmos *A*, *B* y *C* de las siguientes diapositivas.
- 8 Sabiendo que el algoritmo *D* está en $\mathcal{O}(n^2)$, determine la complejidad del método *Calcular*.

Problemas propuestos: algoritmo *Mystery*

```
1.  Algoritmo Mystery(int[] A)
2.      int i = 0
3.      int j = 0
4.      int c = 0
5.      int m = 0
6.      while (i < A.length)
7.          if (A[i] == A[j])
8.              c = c + 1
9.          endif
10.         j = j + 1
11.         if j == A.length
12.             if c > m
13.                 m = c
14.             endif
15.             c = 0
16.             i = i + 1
17.             j = i
18.         endif
19.     endwhile
20.     return m
21. end
```